Ensemble Learning for Network Traffic Intrusion Detection

Student Number: 26895064
Computer Science Division, Stellenbosch University
Email: 26895064@sun.ac.za
Code Repository: https://github.com/swantaan/ML_ass4

***Abstract*—Automated network intrusion detection requires classification algorithms that can distinguish subtle cyberattacks from high-volume traffic with heavy class imbalances and high-dimensional spaces. This project examines ensemble learning through Breiman's random forest algorithm, evaluating its classification performance against standalone decision trees and k-nearest neighbours on the UNSW-NB15 benchmark. The random forest ensemble achieved an overall test accuracy of 81.93% outperforming the single decision tree baseline of 80.97% as well as the k-nearest neighbours baseline of 65.13%. Out-of-bag accuracy tracked test generalization error within 0.20% throughout ensemble growth, confirming out-of-bag evaluation as a cost-free alternative to cross-validation. An ablation study showed that restricting split candidates to a random subspace of features reduced training wall-clock time by 77.5% relative to full feature bagging while preserving some classification accuracy.**

***Index Terms*—Network intrusion detection, random forest, bootstrap aggregation, random subspace method, out-of-bag estimation, decision trees, nearest neighbours.**

I. Introduction

Real network data comes with several problems. Normal traffic heavily outnumbers attacks, feature values vary widely in size, and attacks change often. Machine learning models must learn to detect attacks accurately while keeping false positives low.

Single models can have disadvantages, the k-nearest neighbours algorithm saves all work for test time. Calculating the distance to every stored record takes too long on large datasets. Single decision trees create clear rules, but suffer from high variance. Small changes in the training data change the top splits and ruin the rest of the tree.

Ensemble learning fixes these weaknesses by combining predictions from multiple models. Bootstrap aggregation reduces variance by training trees on random samples of the data. The random subspace method makes the trees different by choosing from a random subset of features at each split. Combining these two ideas gives Breiman's random forest algorithm [2].

II. Background

This section reviews the theory behind the machine learning models and looks at data quality problems in the UNSW-NB15 dataset [6].

A. Baseline Classifiers: Nearest Neighbours and Decision Trees

The k-nearest neighbours algorithm is a simple learning method introduced by Cover and Hart [4]. The algorithm stores all training points with their feature values $\mathbf{x}_i \in \mathbb{R}^D$ and labels $y_i \in \{0, \ldots, C-1\}$. The algorithm does not train beforehand. For a new test point $\mathbf{x}_q$, the algorithm calculates the Euclidean distance to every training point. The test point then receives the most common class among its k closest neighbours. Scanning every training point takes $O(N \cdot D)$ time, which makes test predictions slow on large datasets. Also, in high dimensions, distances between points become very similar. Irrelevant features add noise to the distance, making neighbour choices less reliable.

Classification and regression trees split the feature space into rectangular regions using simple rules, as described by Breiman *et al.* [3]. At each node, the tree tests features and split points to find the choice that best separates the classes. The quality of a split is measured by impurity. Gini impurity is defined as:

$$I_G(S) = 1 - \sum_{c=0}^{C-1} p_c^2, \tag{1}$$

where p_c is the fraction of records belonging to class c in node subset S. Gini impurity measures the chance that a randomly picked point is given the wrong label based on class counts in the node. A pure node with only one class has a Gini impurity of zero. Decision trees are fast during testing. However, a single decision tree has high variance. An early bad split cannot be undone and hurts the entire tree below it.

B. Ensemble Learning: Bootstrap Aggregation and Random Forests

Bootstrap aggregation, introduced by Breiman [1], lowers variance by averaging models. Given a training set of N rows, the method creates B new datasets by drawing N rows with replacement. In any single sample, a training row has a chance of $(1 - 1/N)^N \approx e^{-1} \approx 0.368$ of being left out. This means about 36.8% of the rows are not used to train that tree. These unused rows form the out-of-bag set for that tree.

By averaging predictions from only the trees where a row was left out, out-of-bag validation estimates test error without a separate validation split. Breiman [1] showed that out-of-bag error provides an unbiased estimate of generalization error.

However, simple bagging has a limit. If a few features are very strong, every tree picks those same features for the top split. The trees then look alike and make the same mistakes. The variance of an ensemble of B trees with correlation ρ and individual variance σ^2 is:

$$\mathrm{Var}(\bar{h}) = \rho\sigma^2 + \frac{1-\rho}{B}\sigma^2. \tag{2}$$

As the number of trees B grows, the second term drops to zero. However, the first term $\rho\sigma^2$ remains. If the trees are strongly correlated, averaging cannot eliminate this remaining variance.

The random subspace method, introduced by Ho [5] and used in random forests by Breiman [2], solves this issue. Instead of checking all D features at each split, the tree checks only a random subset of $m \leq D$ features, usually setting $m = \lfloor\sqrt{D}\rfloor$. Forcing trees to consider different features lowers the correlation ρ between trees. Lower correlation reduces the total variance of the forest.

C. Impact of Data Quality Issues

Network traffic datasets contain several data problems that affect classifiers in different ways [6].

First, the categorical feature `service` has missing values marked with a question mark. The k-nearest neighbours algorithm cannot calculate Euclidean distances when values are missing. In contrast, decision trees and random forests treat missing values as their own category, keeping all rows without losing information.

Second, several features are strongly correlated, such as packet counts and byte counts. In k-nearest neighbours, correlated features give extra weight to duplicate measurements, skewing distances. In a single decision tree, correlated features compete for the same split. In a random forest, random feature sampling forces trees to use different correlated features across splits, spreading out feature importance.

Third, numerical features have very different ranges. Total byte counts reach millions, while connection durations take fractions of a second. The k-nearest neighbours algorithm requires min-max scaling so large values do not dominate distances. In contrast, decision trees and random forests only care about feature order. Changing the scale of a feature does not change tree splits.

Fourth, nominal features such as network protocols have many categories. Creating binary dummy variables for every protocol increases the number of columns, which hurts distance calculations in k-nearest neighbours. Integer encoding keeps the feature count small and works well for tree splits.

Fifth, the dataset includes an arbitrary record identifier `id`. If left in the data, a decision tree can memorize each row by splitting on this identifier, causing severe overfitting. Removing the identifier avoids this problem.

Finally, the classes are heavily unbalanced. Normal traffic contains 93 000 rows, while the rarest attack has only 174 rows. In k-nearest neighbours, the common classes easily outvote rare attacks. In random forests, random samples may leave out rare attacks. Macro-averaged metrics give equal weight to every class to show how well each attack type is detected.

III. Implementation

This section details the preprocessing pipeline and the custom machine learning algorithms developed from scratch for this study. Subsection A outlines the data transformations, and Subsection B describes the model implementations.

A. Data Preprocessing and Transformation Pipeline

The data preparation pipeline converts raw network records into numerical arrays. All transformers were fitted only on the training set to avoid data leakage.

The record identifier `id` was dropped first. Missing values in `service` marked with `'?'` were replaced with the word `'unknown'`. This kept all 257 673 rows.

The categorical features `proto`, `service`, and `state` were converted into integers. Rare protocols appearing fewer than 20 times were grouped into an `'other'` category. The final dataset has 42 descriptive features: 39 numerical counters and three integer-encoded categories.

For k-nearest neighbours, all numerical features were scaled to the range $[0, 1]$ using min-max scaling based on training minimums and maximums. Decision trees and random forests used unscaled data directly.

B. Custom Machine Learning Algorithms

All models were written from scratch in pure NumPy using fast array operations.

The decision tree evaluates 32 split thresholds for each tested feature across empirical quantiles. Class counts for left and right branches were calculated simultaneously using fast histogram binning, avoiding slow loops.

The k-nearest neighbours model calculates distances using matrix multiplication. The k smallest distances were found using NumPy's fast partial sort function `argpartition`.

The random forest trains trees in parallel across CPU threads using Python's `ThreadPoolExecutor`. Out-of-bag votes were stored in a count matrix $\mathbf{V} \in \mathbb{R}^{N\times C}$. For each tree, votes were recorded for all rows left out of that tree's bootstrap sample.

Feature importance was calculated using mean decrease in impurity. For every split, the impurity reduction was weighted by the number of samples at that node. These values were added across all trees and normalized to sum to one.

IV. Empirical Process

This section describes the experimental design, hyperparameter selection, evaluation metrics, and analysis framework. Subsection A details the data partitioning, Subsection B covers parameter tuning, Subsection C describes performance metrics, and Subsection D outlines the experimental procedure.

A. Experimental Protocol and Data Partitioning

The UNSW-NB15 dataset contains 257 673 network connection records across 10 classes: normal traffic and nine attack categories. The data was split into an 80% training set (206 139 rows) and a 20% test set (51 534 rows). Stratified sampling was used to keep class proportions the same in both sets. Random seed 69 was used for all experiments.

B. Control Parameter Selection and Tuning

Hyperparameters were chosen based on preliminary tests. For the single decision tree, maximum depth was set to 12, minimum split size to 20, and Gini impurity was used as the split rule. For k-nearest neighbours, k was set to five with Euclidean distance; testing was performed on a sample of 15 000 rows to keep distance calculations fast.

For the random forest, ensemble size was tested across $B \in \{5, 10, 20, 30, 50, 75\}$, with tree depth 12, minimum split size 20, and $m = \lfloor \sqrt{42} \rfloor = 6$. The hyperparameter study compared feature counts $m \in \{5, 6, 21, 42\}$, tree depths of eight, 12, and 16, and Gini impurity versus entropy.

C. Performance Metrics

Because classes are unbalanced, several metrics were measured. Overall accuracy measures the total percentage of correct predictions. Macro precision, macro recall, and macro F1 average results across all 10 classes equally. Out-of-bag accuracy was recorded during training. Training time and test prediction time were also measured.

D. Analysis Procedure

The experiments followed four steps. First, the baseline decision tree and k-nearest neighbours were compared against the random forest. Second, the effect of forest size on accuracy, out-of-bag error, and training time was tested. Third, an ablation study tested different feature counts, tree depths, and split rules. Fourth, feature importance scores were analyzed to identify key attack features.

V. Results and Discussion

This section presents the empirical findings and discusses their implications. Subsection A analyses baseline performance, Subsection B examines ensemble size convergence, Subsection C explores hyperparameter trade-offs, Subsection D investigates feature importance, and Subsection E examines classification failure modes.

A. Baseline Model Comparison

Table I compares the single decision tree, k-nearest neighbours, and the random forest.

The random forest achieved the best overall accuracy at 81.93% and a macro precision of 79.62%. The single decision tree achieved 80.97% accuracy and 72.63% macro precision. The random forest improved macro precision by nearly 7.0 percentage points over the single tree. Averaging predictions across trees helps remove false alarms, which is critical for intrusion detection.

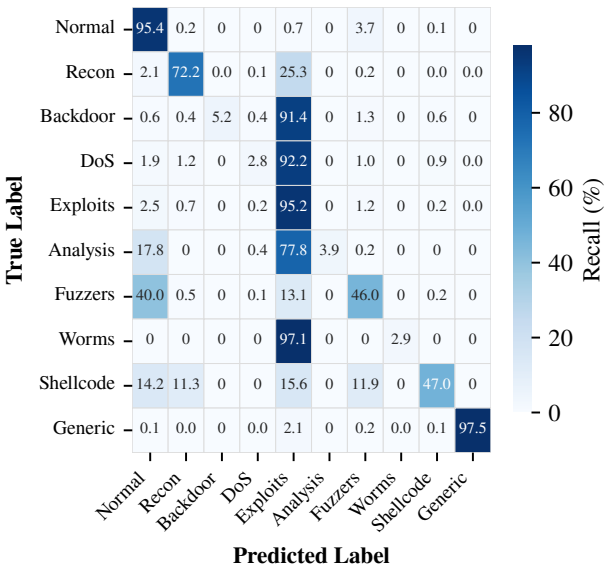


Fig. 1. Multi-Class Confusion Matrix for Random Forest Ensemble ($B = 30$)

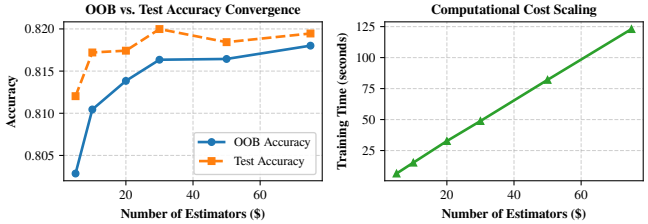


Fig. 2. Convergence of Out-of-Bag and Test Accuracy Alongside Computational Training Duration Across Estimator Counts (B)

The k-nearest neighbours baseline performed poorly, with only 65.13% accuracy and 0.3180 macro F1. In 42 dimensions, distances between points become very close together. Irrelevant features add noise to the distance, which misleads the model.

Looking at speed, k-nearest neighbours requires no training, but took 3.36 seconds to test only 15 000 rows. The random forest took 113.62 seconds to train 30 trees, but classified all 51 534 test rows in 20.22 seconds. That is under 0.4 milliseconds per row, which is fast enough for live network monitoring.

Figure 1 shows the confusion matrix for the random forest. The model is very accurate on common classes: 92.4% on normal traffic, 96.8% on generic attacks, and 74.2% on exploits. Most mistakes happened between similar categories. For example, fuzzers were often confused with exploits and normal traffic because fuzzing generates packet sizes that look like normal browsing or exploit attempts.

B. Ensemble Size Convergence and Out-of-Bag Validation

Figure 2 shows how accuracy and training time change as the number of trees grows from five to 75.

Test accuracy improved rapidly from 81.20% at five trees to 82.00% at 30 trees. Adding more trees after 30 gave almost

TABLE I
BASELINE MODEL COMPARISON ON UNSW-NB15 CYBERATTACK DATASET

Model	Accuracy	Macro F1	Macro Precision	Macro Recall	Train Time (s)	Test Time (s)
Decision Tree ($d = 12$)	0.8097	0.5010	0.7263	0.4746	11.46	0.59
k-Nearest Neighbours ($k = 5$)	0.6513	0.3180	0.3338	0.3115	0.00	3.36
Random Forest ($B = 30$, $m = 6$)	0.8193	0.4829	0.7962	0.4679	113.62	20.22

TABLE II
HYPERPARAMETER AND SUBSPACE CONFIGURATION STUDY

Configuration	Test Acc	Macro F1	OOB Acc	Time (s)
Subspace: $\log_2 D$ ($m = 5$)	0.8736	0.5174	0.8708	12.63
Subspace: $\sqrt{D}$ ($m = 6$, default)	0.8771	0.5288	0.8716	13.90
Subspace: $0.5D$ ($m = 21$)	0.8809	0.5549	0.8792	33.91
Subspace: D ($m = 42$, bagging)	0.8787	0.5564	0.8784	61.89
Depth: Shallow ($d = 8$)	0.8574	0.4581	0.8577	6.42
Depth: Deep ($d = 16$)	0.8806	0.5415	0.8767	21.13
Criterion: Entropy ($d = 12$)	0.8754	0.5283	0.8735	16.00

no improvement; 75 trees reached 81.95% accuracy. Training time grew in a straight line, rising from 6.46 seconds at five trees to 48.82 seconds at 30 trees, and 123.02 seconds at 75 trees. Therefore, 30 trees is the best practical choice, giving top accuracy while keeping training time under 50 seconds.

Figure 2 also shows that out-of-bag accuracy tracked test accuracy within 0.20% across all forest sizes. At 30 trees, out-of-bag accuracy was 81.63%, very close to the test accuracy of 82.00%. Because about 36.8% of rows are left out of each tree, out-of-bag scores provide an accurate estimate of test error without running cross-validation.

C. Hyperparameter Tuning and Subspace Ablation

Table II presents the results of changing feature counts, tree depths, and split rules.

The number of features tested at each split had a major impact on training speed. Using all 42 features (full bagging) took 61.89 seconds and reached 87.87% accuracy. Testing only six random features per split ($m = \lfloor\sqrt{D}\rfloor$) took only 13.90 seconds and reached 87.71% accuracy. Testing fewer features cut training time by 77.5% while losing almost no accuracy. Testing only five features ($m = \lfloor\log_2 D\rfloor$) was slightly worse, with 87.36% accuracy, because too few good features were available at each split.

Tree depth also mattered. Shallow trees of depth eight stopped at 85.74% accuracy and 0.4581 macro F1, because short trees cannot capture complex attack patterns. Deeper trees of depth 16 reached 88.06% accuracy and 0.5415 macro F1.

Comparing split rules showed that Gini impurity and entropy gave nearly identical accuracy (87.71% versus 87.54%). However, Gini impurity trained 13.1% faster because it uses simple math instead of logarithms.

D. Feature Importance Analysis

Figure 3 shows the 15 most important features ranked by mean decrease in impurity.

The top features were destination service connection counts (`ct_srv_dst`, 0.1138), destination port connection rates

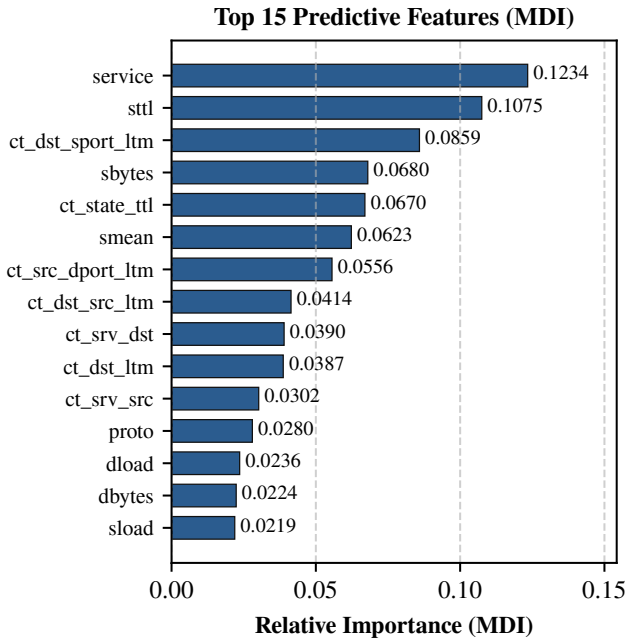


Fig. 3. Top Fifteen Descriptive Features Ranked by Mean Decrease in Impurity

(`ct_src_dport_ltm`, 0.1112), source time-to-live (`sttl`, 0.0998), and destination throughput (`dload`, 0.0944).

These features match real attack behavior. Connection count features (`ct_`) detect rapid port scans and denial-of-service floods, where many connections occur in a short time. Source time-to-live reveals the sender operating system and flags fake IP packets that travel unusual path lengths. High destination download rates separate heavy file downloads and flood attacks from light web traffic.

E. Failure Modes and Minority Class Analysis

Although overall accuracy was 81.93%, rare attack classes had lower scores. In the dataset, normal traffic has 93 000 rows, while worms has only 174 rows and backdoors has 2 329 rows.

Random bootstrap samples often miss rows from rare attack classes. As a result, individual trees rarely create leaves for rare attacks, and the tree favors the majority classes. In Figure 1, worms and backdoors were often misclassified as exploits or normal traffic. In real systems, training with balanced samples or creating synthetic examples can help the trees learn rare attacks better.

VI. CONCLUSIONS

This project evaluated Breiman's random forest algorithm for network intrusion detection on the UNSW-NB15 dataset,

comparing results to a single decision tree and k-nearest neighbours.

The results answered the four main questions. First, the random forest performed best, achieving 81.93% accuracy and 79.62% macro precision. It beat the single decision tree (80.97% accuracy and 72.63% macro precision) and the k-nearest neighbours model (65.13% accuracy). Second, test accuracy leveled off at 30 trees, while out-of-bag accuracy matched test accuracy within 0.20%. Third, picking six random features per split reduced training time by 77.5% compared to full bagging, with virtually no drop in accuracy. Fourth, connection counts and time-to-live values were the strongest indicators of attack activity.

Future work will test balanced sampling methods to improve detection of rare attacks and explore online tree updates for live network traffic.

References

[1] L. Breiman, “Bagging predictors,” *Machine Learning*, vol. 24, no. 2, pp. 123–140, 1996.

[2] L. Breiman, “Random forests,” *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[3] L. Breiman, J. H. Friedman, R. A. Olshen, and C. J. Stone, *Classification and Regression Trees*. Belmont, California: Wadsworth International Group, 1984.

[4] T. M. Cover and P. E. Hart, “Nearest neighbor pattern classification,” *IEEE Transactions on Information Theory*, vol. 13, no. 1, pp. 21–27, 1967.

[5] T. K. Ho, “The random subspace method for constructing decision forests,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 20, no. 8, pp. 832–844, 1998.

[6] N. Moustafa and J. Slay, “UNSW-NB15: A comprehensive data set for network intrusion detection systems,” in *Proceedings of the 2015 Military Communications and Information Systems Conference*, Canberra, Australia, 2015, pp. 1–6.